

ANNEXURE APPLICATION-BASED QUESTIONS

Neural Networks, Genetic Algorithms, and Model Evaluation

Name:	THOUHID
Register Number:	192421280
Department :	IT
Course / Subject:	ITA0609-ML

Annexure A – Application-Based Questions

Question 1: Backpropagation in a Multilayer Perceptron (5 Marks)

A multilayer perceptron (MLP) used for binary classification typically consists of an input layer, one or more hidden layers, and an output layer with a single neuron (sigmoid activation) that produces a probability between 0 and 1. Backpropagation trains this network by propagating the prediction error backward through the layers and adjusting the weights to reduce that error.

Step 1: Network Setup (Example)

Consider a simple MLP with 2 input neurons, 1 hidden layer with 2 neurons, and 1 output neuron, using the sigmoid activation function at every layer.

Parameter	Value
Inputs (x1, x2)	0.5, 0.8
Weights (input → hidden)	w11=0.2, w12=0.4, w21=0.3, w22=0.1
Weights (hidden → output)	w1=0.5, w2=0.6
Target Output (y)	1
Learning Rate (η)	0.1

Table 1. Sample weights and inputs used to illustrate one training pass

Step 2: Forward Pass

Each hidden neuron computes a weighted sum of the inputs plus bias, then applies the sigmoid function $\sigma(z) = 1 / (1 + e^{-z})$ to produce its activation. The output neuron repeats this process using the hidden layer's activations as its inputs, producing the network's predicted output $\hat{y}$.

$$h1 = \sigma(w11 \cdot x1 + w21 \cdot x2) = \sigma(0.2 \times 0.5 + 0.3 \times 0.8) = \sigma(0.34) \approx 0.584$$

$$h2 = \sigma(w12 \cdot x1 + w22 \cdot x2) = \sigma(0.4 \times 0.5 + 0.1 \times 0.8) = \sigma(0.28) \approx 0.570$$

$$\hat{y} = \sigma(w1 \cdot h1 + w2 \cdot h2) = \sigma(0.5 \times 0.584 + 0.6 \times 0.570) = \sigma(0.634) \approx 0.653$$

Step 3: Error Calculation

The error is measured using a loss function such as mean squared error or binary cross-entropy. Using squared error for simplicity:

$$E = \frac{1}{2}(y - \hat{y})^2 = \frac{1}{2}(1 - 0.653)^2 \approx 0.060$$

Step 4: Backward Propagation of Errors

The gradient of the error with respect to each weight is found using the chain rule, starting at the output layer and moving backward. The output error term is $\delta_{\text{output}} = (\hat{y} - y) \cdot \hat{y} \cdot (1 - \hat{y})$. This term is then propagated to each hidden neuron as $\delta_{\text{hidden}} = \delta_{\text{output}} \cdot w \cdot h \cdot (1 - h)$, where w is the corresponding output-layer weight. Each weight is updated using: $w_{\text{new}} = w_{\text{old}} - \eta \cdot \delta \cdot (\text{input to that weight})$.

$$\delta_{\text{output}} = (0.653 - 1) \times 0.653 \times (1 - 0.653) \approx -0.0787$$

$w1_{\text{new}} = 0.5 - 0.1 \times (-0.0787) \times 0.584 \approx 0.5046$, and similarly for $w2$, $w11$, $w12$, $w21$, $w22$ using their respective δ terms.

Step 5: Effect of Learning Rate and Activation Function

A learning rate that is too high causes the weight updates to overshoot the minimum of the error surface, producing unstable or oscillating training, while a rate that is too low leads to very slow convergence and a risk of getting stuck in flat regions. A moderate learning rate, often tuned experimentally or scheduled to decay over epochs, gives the best balance of speed and stability.

The choice of activation function similarly affects convergence: sigmoid and tanh can suffer from vanishing gradients in deep networks because their derivatives shrink for large inputs, slowing learning in early layers. ReLU and its variants avoid this for positive inputs and generally converge faster, though they can suffer from inactive ('dead') neurons. The output activation must also match the task — sigmoid for binary classification, softmax for multi-class problems.

Final Answer: Backpropagation updates weights by computing the output error, propagating scaled error terms backward through each layer using the chain rule, and adjusting every weight in the direction that reduces the overall error, with the learning rate and activation function together governing how quickly and reliably the network converges.

Question 2: Genetic Algorithm for Maximizing a Fitness Function (5 Marks)

A Genetic Algorithm (GA) searches for an optimal solution by evolving a population of candidate solutions across generations using selection, crossover, and mutation. Consider a population of candidate solutions described by two parameters, Speed and Accuracy (each scaled 0–10), evaluated as suitable (Yes) or not suitable (No) based on a fitness function $\text{Fitness} = \text{Speed} + \text{Accuracy}$.

Step 1: Initial Population (Generation 0)

Chromosome	Speed	Accuracy	Fitness (Speed + Accuracy)	Suitable?
C1 (0110)	6	4	10	No
C2 (1001)	5	7	12	No
C3 (1110)	7	6	13	Yes
C4 (0101)	4	8	12	No

Table 2. Initial randomly generated population and their fitness values

Step 2: Selection

Selection retains individuals with higher fitness so that their genetic material is more likely to appear in the next generation. Using roulette-wheel (fitness-proportionate) selection, C3 (fitness 13) and C2/C4 (fitness 12) are the most likely to be chosen as parents, while C1 (fitness 10) is less likely to be selected.

Selected parents for reproduction: C3 and C4 (highest combined fitness).

Step 3: Crossover

Crossover combines the genetic material of two parent chromosomes to produce offspring, mixing traits from each. Using single-point crossover on C3 (1110) and C4 (0101) at the midpoint:

Offspring 1 = 1101 → Speed 6, Accuracy 8 → Fitness 14

Offspring 2 = 0110 → Speed 5, Accuracy 6 → Fitness 11

Step 4: Mutation

Mutation randomly flips a bit in a chromosome to introduce diversity and helps the algorithm explore parts of the hypothesis space that crossover alone might miss, reducing the risk of premature convergence to a local optimum. Applying a small mutation probability to Offspring 1 (1101 → 1111) increases both Speed and Accuracy to 7 and 8:

Mutated Offspring 1 = 1111 → Speed 7, Accuracy 8 → Fitness 15

Step 5: Next Generation and Convergence

Chromosome	Speed	Accuracy	Fitness	Suitable?
C3 (retained)	7	6	13	Yes

Mutated Offspring 1 (1111)	7	8	15	Yes
Offspring 2 (0110)	5	6	11	No

Table 3. Generation 1 population after selection, crossover, and mutation

Across generations, individuals with low fitness (such as C1) are gradually eliminated while high-fitness traits accumulate in the population through repeated selection, crossover, and mutation, driving the population toward the region of the search space that maximizes the fitness function.

Final Optimized Solution: Chromosome 1111 (Speed = 7, Accuracy = 8, Fitness = 15) emerges as the best candidate solution after evolution, representing the optimal balance of Speed and Accuracy for this fitness function.

Question 3: Neural Network Classification for Disease Prediction (5 Marks)

A neural network can be trained to predict whether a patient has a disease based on attributes such as Age, Blood Pressure, Glucose Level, and Lifestyle Risk. The categorical attributes (Blood Pressure, Glucose Level, Lifestyle Risk) are first encoded numerically, the features are normalized, and the network is trained using backpropagation with a sigmoid output neuron that classifies the outcome as Yes (disease present) or No (disease absent).

Dataset

Example	Age	Blood Pressure	Glucose Level	Lifestyle Risk	Actual Output	Predicted Output
1	45	High	High	High	Yes	Yes
2	50	Normal	High	Medium	Yes	No
3	30	Normal	Normal	Low	No	No
4	60	High	High	High	Yes	Yes
5	40	Normal	Normal	Medium	No	Yes

Table 4. Patient dataset with actual and predicted disease outcomes

Step 1: Training with Backpropagation

The network takes the four (encoded) attributes as input, passes them through one or more hidden layers with a non-linear activation function, and produces a predicted probability of disease at the output layer. During training, the prediction is compared with the actual label, and backpropagation adjusts the weights across epochs to minimize the classification error, similar to the process described in Question 1.

Step 2: Confusion Matrix (TP, TN, FP, FN)

Comparing the Actual and Predicted columns for each example:

Example	Actual	Predicted	Result
1	Yes	Yes	True Positive (TP)
2	Yes	No	False Negative (FN)
3	No	No	True Negative (TN)
4	Yes	Yes	True Positive (TP)
5	No	Yes	False Positive (FP)

Table 5. Classification of each example into TP, TN, FP, or FN

Metric	Value
True Positives (TP)	2
True Negatives (TN)	1
False Positives (FP)	1
False Negatives (FN)	1

Step 3: Performance Evaluation

Accuracy = (TP + TN) / Total = (2 + 1) / 5 = 0.60 → 60%

Precision = TP / (TP + FP) = 2 / 3 ≈ 0.667 → 66.7%

Recall (Sensitivity) = TP / (TP + FN) = 2 / 3 ≈ 0.667 → 66.7%

F1-Score = $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) \approx 0.667 \rightarrow 66.7\%$

With only 5 examples, the model correctly classifies 3 out of 5 patients. The single false positive (Example 5) and false negative (Example 2) indicate the model is currently only moderately reliable and would benefit from more training data.

Step 4: Role of Preprocessing and Feature Selection

Preprocessing steps such as encoding categorical values (e.g., Low/Medium/High → 0/1/2), normalizing numeric attributes like Age so that no single feature dominates due to scale, and handling missing or noisy values all help the network learn more stable and meaningful weight updates. Feature selection removes irrelevant or redundant attributes, reducing overfitting risk and training time while helping the model focus on the attributes (e.g., Glucose Level, Blood Pressure) that are most predictive of disease presence, ultimately improving both accuracy and reliability.

Final Answer: The model correctly classifies 3 of 5 patients (TP=2, TN=1, FP=1, FN=1), giving an accuracy, precision, recall, and F1-score of approximately 66.7% (accuracy 60%). Proper preprocessing and feature selection are essential to improve this performance further.

Overall Summary

Question	Key Result	Recommended Takeaway
1	Forward pass $\hat{y} \approx 0.653$; weights updated via backpropagated error terms	Tune learning rate and activation choice for stable convergence
2	Best chromosome: 1111 (Speed=7, Accuracy=8, Fitness=15)	Selection, crossover, and mutation together drive convergence to the optimum
3	TP=2, TN=1, FP=1, FN=1; Accuracy=60%, F1≈66.7%	Improve via preprocessing, feature selection, and more training data